DevOps Project Report for Group L

DevOps, Software Evolution and Software Maintenance - BSDSESM1KU

Andreas Bartholdy Christensen anbc@itu.dk
Marcus Andreas Aandahl maraa@itu.dk
Villads Grum-Schwensen vilg@itu.dk
Markus Brandt Højgaard mbrh@itu.dk
Lukas Brandt Pallesen lupa@itu.dk

1 System's Perspective

1.1 The onion architecture

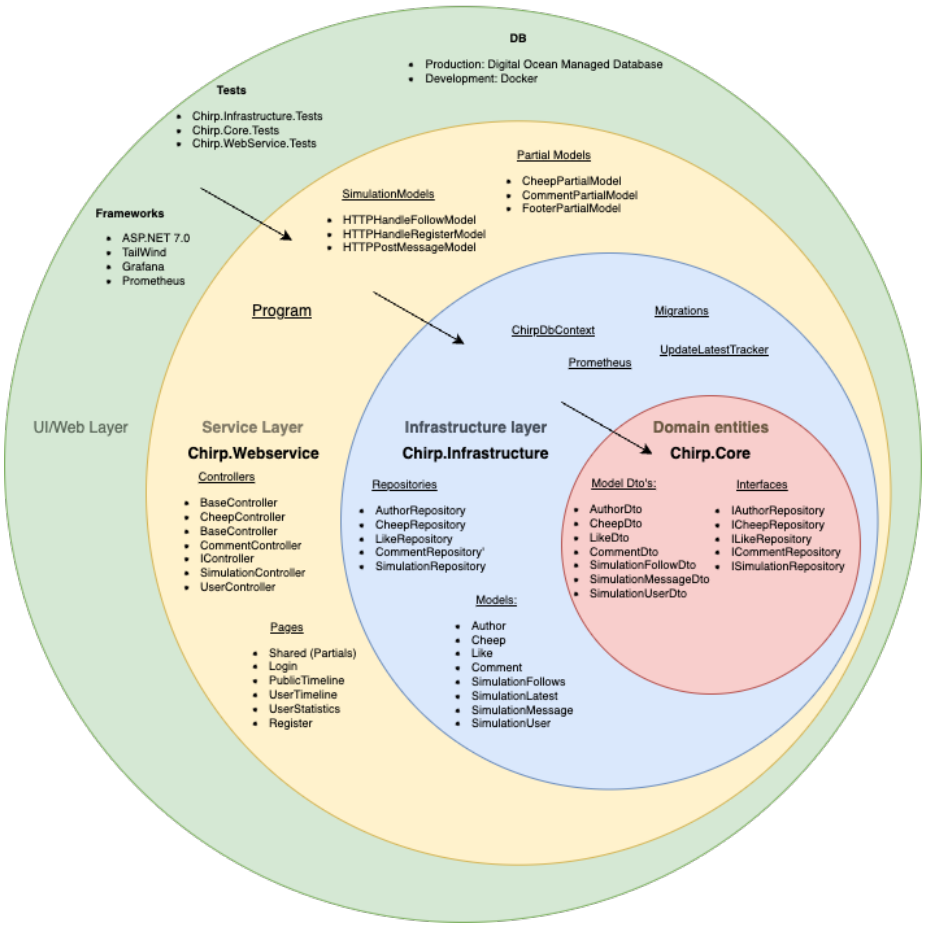


Figure 1: Onion Architecture

We chose to work with the onion architecture. The architecture provides loose coupling with the dependencies only flowing inwards. The interactions between the layers are facilitated through interfaces and abstractions, hence promoting loose coupling.

1.2 The layers

- The Domain/Core layer
  - `DTOs` (Data Transfer Objects) to transfer data between layers in the system

  - `Interfaces` of the `repositories` to promote loose coupling
- The Infrastructure Layer
  - `Repositories` for data access with the Entity Framework in the `DbContext`
  - `Models`as the entities used in the `DbContext` on the same layer
  - Additional services such as `Prometheus` and `UpdateLatestTracker` for monitoring the system
- The Webservice/Presentation layer
  - `Controllers` to handle HTTP request and responses with access to the `repositories`
  - `Pages` that contains UI components to render the data to the user. In our project they're referred to as Partials
- Web layer
  - The web layer is outside of the main project and represents external dependencies such as `Frameworks`, `Tests` and the `Database`

1.3 Dependencies

We have utilized a range of tools. For coding and development, we have used: C# .NET and PostgreSQL for base functionality. We have used Tailwind CSS for UX/UI improvements. To ensure test-driven-development we have used Playwright and Moq in our test-suite.

For version control we have used GitHub.

We have hosted our service at Digital Ocean, and done CI/CD through GitHub actions.

In addition, we also have the following dependencies:

- Docker
- DockerHub
- Vagrant
- Serilog
- Grafana
- Prometheus
- Loki
- AspNetCoreAnalyzers
- CSharpier
- EF Core
- OpenTelemetry

1.4 Interactions of subsystems

1.5 State of systems

The overall state of our system is good. We have been using SonarCloud to quantify the state of our system, and here we pass the quality gate. Although

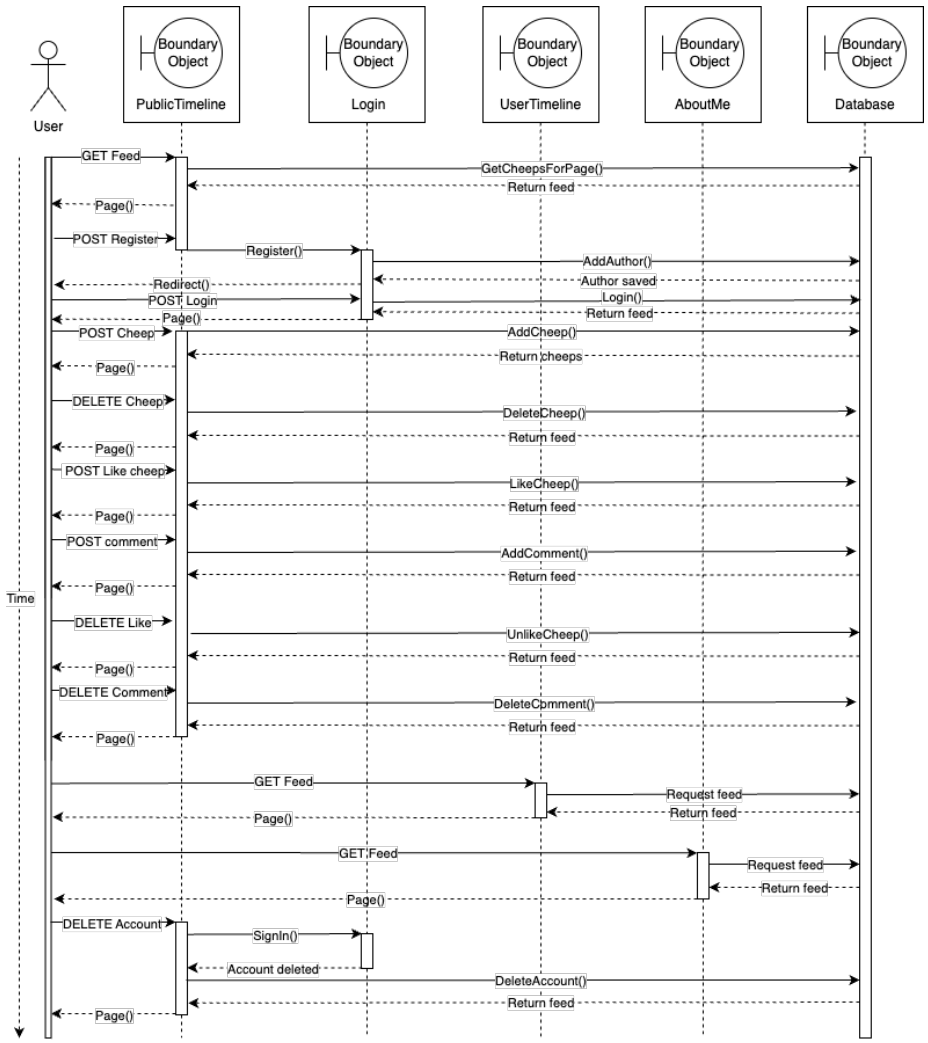


Figure 2: User Sequence

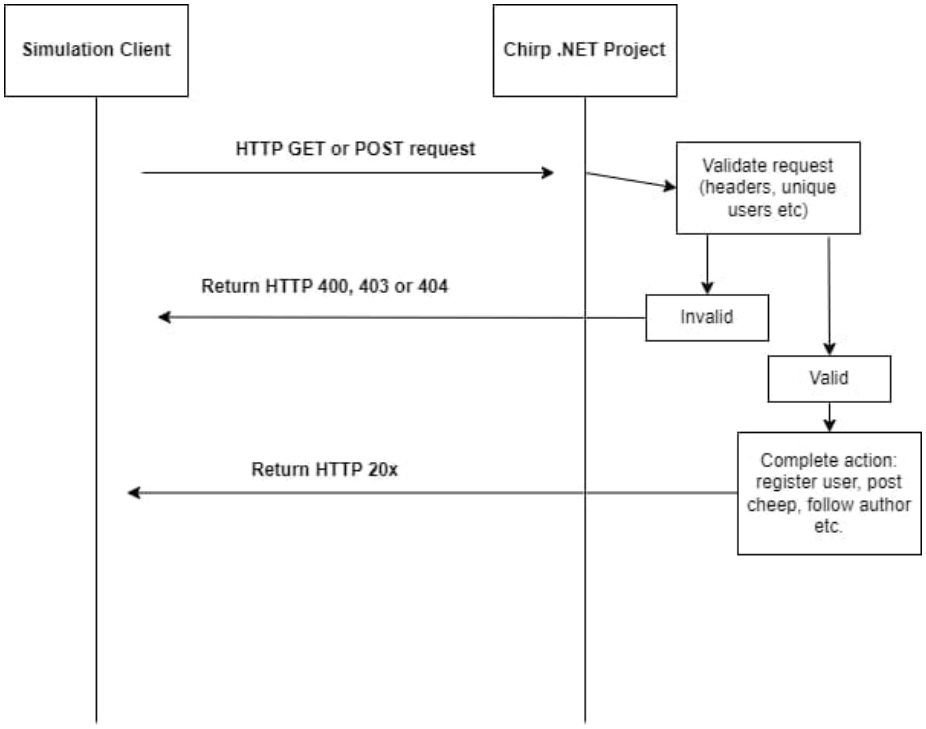


Figure 3: Sequence Diagram

this comes with a caveat. Estimations from SonarCloud is that we have an estimated technical debt of 1 day and 3 hours. We have 95 issues spanning the following qualities:

- Security (11 issues)
- Reliability (18 issues)
- Maintainability (88 issues)

Therefore, if this project was to continue it would be preferable to address some of these, which could prevent vulnerabilities and increase the overall quality of the system.

2 Process' perspective

2.1 CI/CD Chain

2.1.1 CI Chain

Our CI Chain can be split into the following 3 workflows, where the latter 2 can be considered our main CI chains:

- Dependabot
- Sonarcloud
- Code coverage Chain

2.1.1.1 Dependabot Dependabot is used to, on a weekly basis, make us aware of dependencies which have new versions, such that we stay up-to-date with our dependency ecosystem. It creates a PR for each out-of-date dependency, which we can then choose to accept or decline. This is a built-in bot provided by Github.

2.1.1.2 Sonarcloud SonarCloud is a tool which enables us to get an analysis of our project. When a PR is created, an analysis is added as a sticky comment, which showcases potential issues with the implemented code, along with new potential security flaws. It is also run on our default branch, which gives us an general analysis of the project.

2.1.1.3 Code coverage CI Chain The code coverage CI chain attempts to build the project, then runs the tests whilst creating a test coverage report. It is run on every created pull request to our `itu-minitwit` branch.

It has the following steps:

This chain ensures that we can build the project, our tests pass, and that enough of the code is tested at all times. Furthermore, it is also used as a reminder for the developers to write tests to their newly implemented features.

Step Nr.	Step Title	Step Description
1	Checkout	Checks out the latest git commit for the given PR, so that it can be used later in the chain.
2	Set up .NET	Sets up .NET so that the project can be set up, built and run.
3	Restore dependencies	Restores the nuget packages which are the project's dependencies.
4	Restore tools	Restores the .NET tools the project uses (such as `pwsh` to run install playwright).
5	Build the project	Builds the project. Csharpier linter is also run. If it fails, the CI chain fails, and thus the Github PR check fails.
6	Install playwright (required for E2E tests)	Installs playwright, which is used for end-to-end tests.
7	Run tests with coverage	Runs tests. If it fails, the CI chain fails, and thus the Github PR check fails. The tests are run using the `dotcover` tool, which generates a code coverage report, using the `dotcoverConfig.xml` as configuration.
8	Report Generator	Takes the dotcover-generated `analysis.xml` report, and transforms it to a `Cobertura` format that can be used to create a PR message showcasing the code coverage.
9	Build Code Coverage Summary Report	Takes the `Cobertura` report, and converts it to a markdown format, which can be used to showcase the code coverage as a PR comment.
10	Add Coverage PR Comment	This step takes the generated markdown-format code coverage report, and adds it as a sticky PR comment.
11	Write to Job Summary	This step takes the generated markdown-format code coverage report, and adds it as a summary to the given workflow.
12	Test Check Failure of Code Coverage Summary Report	Takes the code coverage report, and given a minimum threshold of 60%, fails the check if the code coverage % is below 60%. If between 60% and 80%, only a warning is given.

Figure 4: Code Coverage Chain

2.1.1.4 CI Chain reflections

Looking back at it now, we should have been better at utilising and respecting our CI chains. This is mainly due to use prioritising other tasks, which ended up taking most of the time we allocated to the project.

Although setting up Dependabot, we weren't good at continuously updating the recommended dependencies, which we should have done as a weekly ritual.

Furthermore, early on in the project, we had commented out our 2 primary CI chains during a major refactoring of the project, and had forgotten to uncomment it. Looking back at it, we should have never uncommented them, but rather just adjusted to them, as it could have led to some serious consequences.

2.1.2 CD Chain

Our CD Chain can be split into the following 2 workflows:

- Publish App Chain
- Continuous Deployment Chain

2.1.2.1 Publish App CD Chain

Our publish app CD chain is run on a schedule, which is every Thursday. It is responsible for creating a minor release once a week.

It can be split into the `prepare release` and `release app` jobs, where the latter job runs as a matrix for the OS' `linux-x64`, `win-x64` and `osx-x64`. It has the following steps:

Step Nr.	Step Title	Step Description
1	Checkout (prepare)	Checks out the given commit history for the `itu-minitwit` branch. It does so with a `fetch-depth: 0` to ensure we get the history of all tags, which are used later in the chain.
2	Determine new version tag (prepare)	Gets the latest tag, and adds 1 to the minor version, to create the new future tag.
3	Checkout (release)	Checks out the latest git commit for the given PR, so that it can be used later in the chain.
4	Set up .NET (release)	Sets up .NET so that the project can be set up, built and run.
5	Build and publish (release)	Builds and publishes the project to a zip file given the current OS in the matrix. Csharpier linter is also run on build.
6	Create and push new tag (release)	Creates the new tag given the `Determine new version tag` step.
7	Publish app (release)	Generates a release if not yet existent given the new tag, and adds the generated zip files to it.

Figure 5: Publish Chain

This ensures that we continuously deliver releases.

2.1.2.2 Continuous Deployment CD Chain

In Digital Ocean, we have already provisioned 2 VMs, along with 2 reserved IPs pointing to them, following

the green-blue deployment model. Our CD chain is responsible for updating our VMs in Digital Ocean, and switch the reserved IPs in order to minimise downtime during deployments. This CD chain is run on every push to the `itu-minitwit` branch, and can be triggered manually.

It has the following steps:

Step Nr.	Step Title	Step Description
1	Checkout	Checks out the latest git commit for the given PR, so that it can be used later in the chain.
2	Login to Docker Hub	Given secrets in our github repository, logs in to Docker Hub, so that we later can push our image to our repository.
3	Set up Docker Buildx	Sets up Docker Buildx.
4	Build and push minitwitimage	Builds an image from our Dockerfile, and pushes it to our repository on Docker Hub, along with the tag `latest` .
5	Configure SSH	Adds the SSH key, which is stored in our github repository's secrets, to our working directory. This is so we can SSH to our VM later in the chain.
6	Deploy to server	We SSH into our green droplet/VM, and run the `deploy.sh` script, which pulls our newly created image, and re-runs the container through docker-compose.
7	Get blue droplet id	We fetch the blue droplet id, so we later can use it to reassign our reserved IPs.
8	Get green droplet id	We fetch the green droplet id, so we later can use it to reassign our reserved IPs.
9	Un-assign green IP	We un-assign the reserved IP pointing to our green droplet, so we later can re-assign to our blue droplet.
10	Wait 30secs	We wait 30 seconds whilst Digital Ocean finishes un-assigning the reserved IP.
11	Reassign blue IP to green droplet	We reassign the blue reserved IP to the green droplet, which is running with our latest Docker image and becomes our new blue droplet.
12	Wait 30secs	We wait 30 seconds whilst Digital Ocean finishes reassigning the reserved IP.
13	Reassign green IP to blue droplet	We reassign the green reserved IP to the old blue droplet, which is now our new green droplet.

Figure 6: CD Chain

In short, when we start a deployment, we:

- Build and upload the new docker image to Docker Hub
- SSH into our green droplet
- Update the Docker image and restart using docker-compose
- Switch reserved IPs

This ensures that we continuously deliver the latest version of our application, whilst minimising our downtime during deployment thanks to our green-blue deployment model. In this scenario, we decided to use Digital Ocean's API to make the switch between green and blue droplets, and have used the reserved IPs as our router. Furthermore, with this CD pipeline, we can also easily can go back to the "non-live" droplet by switching the reserved IPs, in case the live droplet is somehow erroneous.

The main potential weakness we see with this CD chain, is in case 2 deployments are started in quick succession. In that case, we don't know how the deployments will interact.

For a more robust CD chain, we should probably ensure that only one deployment can run at once. However, due to time constraints, we weren't able to implement this.

2.2 Monitoring

2.2.1 How do we monitor

We do pull-based monitoring, utilizing Prometheus and Grafana. Grafana pulls from Prometheus, which again pulls from our application.

2.2.2 What do we monitor

Our monitoring is passive blackbox. We monitor the following in our application:

- The duration of HTTP requests
- Average duration of HTTP requests
- What endpoints are being hit
- What HTTP response is returned

2.3 Logging

2.3.1 How do we log

We use the .NET log framework Serilog to collect custom as well as system logs from our program. Serilog is a simple way to send logs to a specified output. In our case the collected logs are outputted to Loki, which is running as a container. Loki acts as a log aggregator, which efficiently indexes the logs only based on its metadata, instead of the whole log text, which makes logs available milliseconds after they are received. We then use Grafana to visualize the collected logs from Loki.

2.3.2 What do we log

Per default we log everything from 'log level' information. This is to ensure, that in case something goes wrong while live, we can get access to all necessary information as normally displayed in the terminal. Furthermore, we have also implemented some custom log prints, which prints whenever a controller is called, to help us trace user behavior.

2.4 Security

From the security assessment we realized that our ports on different containers were public. This is a theoretical security issue since software such as Grafana uses authentication. We felt this was not a major risk.

Another reflection was on using SSH keys. This enabled us to not send password through network traffic, thereby reducing risk of interception. Our project was also layered with SSL while hosted on Digital Ocean.

In general, our simulation endpoints were 'protected' with an auth-header. Sensitive information such as configuration secrets were ensured to not be committed to VCS. User passwords stored in database were encrypted with SHA256.

2.5 Strategy for scaling and upgrades

2.5.1 Scaling strategy

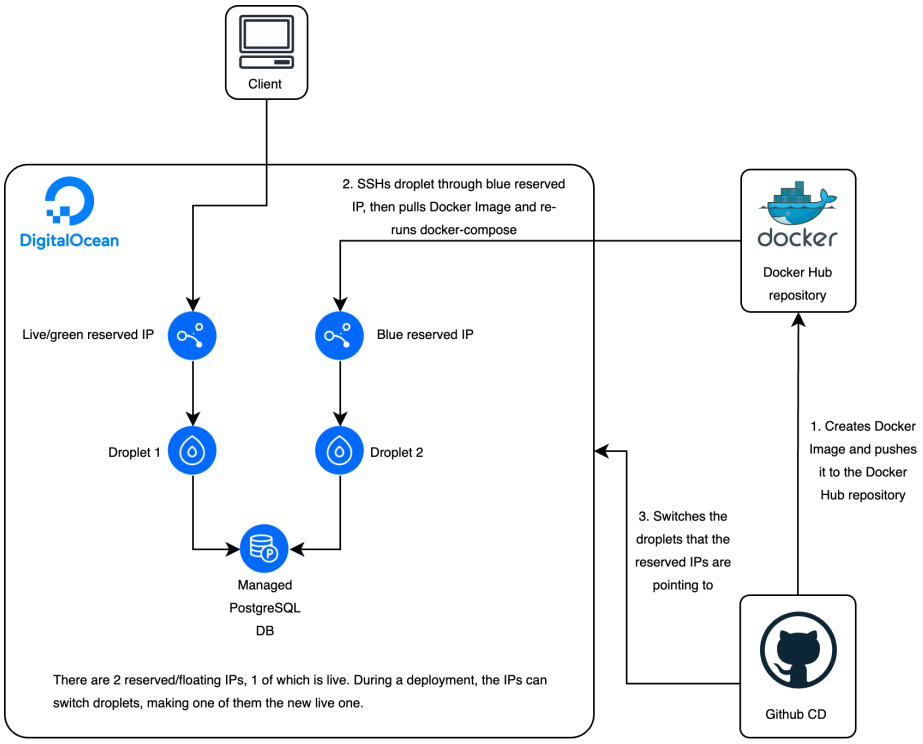
When it comes to scaling, we didn't have enough time to implement a solution. However, we did still make some considerations regarding this topic.

The first thing we would have done would be to implement some kind of horizontal scaling strategy. We wanted to do this with Docker Swarm, where on our droplet, we could orchestrate container replication, along with utilising load balancing.

This would have improved the availability of our application. However, it might have also caused some problems, as we would've started building a distributed system. This would have likely led us to have issues with having to synchronise our logging and monitoring tools across our container replicas and our load balancer.

2.5.2 Upgrades

Regarding which upgrades we made to our project, one notable one was the implementation of the green-blue deployment model. The diagram below shows our setup in Digital Ocean, along with how our toolchain interacts.



The way we implemented green-blue, was by having 2 droplets at all times, with 2 reserved/floating IPs pointing to them. One of those IPs is pointing to our live droplet (green), whilst the other one is pointing to a non-live one (blue). Furthermore, the droplets also share the same Managed PostgreSQL DB.

Our CD chain ensures that the deployment is applied to the droplet to which our non-live reserved IP is pointing to. Thereafter, the CD chain switches the reserved IPs, such that the live reserved IP now pointing to our previously non-live droplet, and the non-live reserved IP is now pointing to our previously live droplet. This repeats on every deployment.

The objective with this setup is to ensure very low, to no downtime during deployments. In case we find out our live droplet is erroneous, this also allows us to switch back to a our previously live droplet.

2.6 Reflection on AI assistance

During the project we've used ChatGPT to search for information and code snippets for guidance. It is a mutual opinion in the group that copy-pasting our code to any chat bot or other AI tools should be avoided as we're unaware of where the data is stored and exactly who has access. Therefore we view AI tools, such as ChatGPT, as an addition to Google where we can search for information and get answers to our questions. Additionally, manual development has the

best learning outcome.

3 Lessons Learned Perspective

3.1 Continuous refactoring

At the beginning of the simulation, we implemented functionality for receiving HTTP requests and storing data in separate tables unrelated to 'real' data. This data served the relevant HTTP simulation endpoints, but was purposely not visible on our applications feed. Later on we then had to display this simulation data, which initially prompted us to change how we processed incoming requests. However this would require a migration, which we feared could cause a wipe of our database. Therefore we had to figure out a way to transform incoming data into visible data, without changing data-models (and causing a migration). Ultimately we made it possible to transform this data into feed-viable data, without changing models. See commits `ec8c79d2` and `39a2df11`.

3.2 Operation

As mentioned above we have continuously refactored our system throughout this project which sometimes has led us to having crashes or bugs in our live system. This has truly shown us the value of having monitoring and logging setup for our systems. Logging would give us the benefit of being able to see stack traces and logs, and monitoring would give us valuable insight into the state of users experience with our system. Many resources have been used setting up these tools, and therefore we unfortunately have not gotten to use the data from these as much as we might have wanted to, but we can see their value and how we would rely on these if we were to move forward with this project.

3.3 Maintenance

During the start of the project, we experienced short downtimes on our droplet when we deployed a new version of our system. With the implementation of Green-Blue we've succeeded with deployment with minimized downtime, which will enhance user experience and monitoring.

During the project we've had more focus on operations rather than development. Compared to previous projects, our primary focus was to establish foundations to make DevOps possible. We've created a Blue-Green cloud solution to minimize downtime. Monitoring and logging was implemented to keep track of the system after deployment, and CI/CD Chain to ensure quality and avoid potential failures when implementing new features.